

Universidad ORT Uruguay
Facultad de Ingeniería

Obligatorio 1

Entregado como requisito para la materia
Sistemas Operativos

Sebastian Quiroz — 323189
Juan Manuel Reolon — 331598

Docentes: Santiago Gonzalez y Renzo Chiarino

2026

Índice

1. Parte 1	3
1.1. Ticket 104	3
1.2. Ticket 105	3
1.3. Decisiones tomadas	4
2. Parte 2	4
2.1. Ejercicio 1	4
2.1.1. Contexto del problema	4
2.1.2. Ejecución	4
2.2. Ejercicio 2	4
2.2.1. Grafo de precedencias	4
2.2.2. Inicialización de los semáforos	5
3. Parte 3	6
3.1. Idea inicial	6
3.2. Construcción del código	6
3.3. Inteligencia Artificial	6
3.4. Ejemplo de una tarea	6
4. Parte 4	7
4.1. Elección de imagen base	7
4.2. Análisis de seguridad	7
4.3. Configuración de seguridad aplicada	7
4.4. Decisiones de diseño	9
4.5. Inteligencia Artificial	9
4.6. Ejecución	9

1. Parte 1

En este ejercicio se nos dio un código de bash y debíamos investigar para corregir dos problemas:

- 1) **Informe de Autopsia (Diagnóstico):** Explica técnicamente por qué ocurrieron los incidentes 104 y 105. Debes identificar la línea exacta del fallo y explicar qué hizo el intérprete de Bash (conceptos de Word Splitting, Globbing y expansión de variables).
- 2) **El Parche:** Entrega el script modificado (`paddock_manager_fixed.sh`) aplicando buenas prácticas de scripting (quoting, validación de variables vacías y manejo seguro de rutas) para que sea robusto ante errores humanos

1.1. Ticket 104

El problema está en la línea:

```
grep $PARAMETRO $ARCHIVO_CSV
```

Al correr esa línea, como `PARAMETRO` está sin comillas, Bash aplica word splitting. Entonces el comando real queda como si fuera:

```
grep Gorra Edicion Especial Monza inventario_f1.csv
```

O sea, intenta buscar Gorra, pero interpreta a Edicion, Especial y Monza como archivos donde debe buscar, además de `inventario_f1.csv`. Por eso aparece el error de que Edicion no existe; no significa que no exista un objeto llamado Edicion, sino que Bash interpreta que se está intentando buscar Gorra en un archivo llamado Edicion.

La corrección sería usar comillas:

```
grep "$PARAMETRO" "$ARCHIVO_CSV"
```

<https://stackoverflow.com/questions/28958471/grepping-for-a-sentence-from-inside-a-bash-script>

1.2. Ticket 105

El problema está en la línea:

```
rm $DIR_MERCADERIA/$PARAMETRO*.txt
```

El error ocurre porque la variable `PARAMETRO` no fue validada antes de su uso.

Cuando el script se ejecuta sin argumento:

```
./paddock_manager.sh descatalogar
```

entonces:

```
$PARAMETRO=""
```

La línea se convierte en:

```
rm $DIR_MERCADERIA/*.txt
```

Esto es expandido automáticamente por Bash a todos los archivos `.txt` del directorio `/mercaderia/`. El comando ejecutado termina siendo equivalente a:

```
rm ./mercaderia/*.txt
```

1.3. Decisiones tomadas

Para el ejercicio 4, se pedía presentar los tres ejercicios con contenedores, para el correcto funcionamiento de este ejercicio era necesario agregar los archivos por los cuales opera, por esto mismo decidimos agregar `inventario_f1.csv` y la carpeta `/mercadería`. La creación de las mismas fue utilizando inteligencia artificial.

2. Parte 2

2.1. Ejercicio 1

2.1.1. Contexto del problema

En este ejercicio, notamos la similitud con el problema dado en clase llamado Productor/Consumidor, por esto mismo empezamos a construir la estructura manteniendo la idea de este problema. Utilizamos los semáforos para poder saber si la barra estaba liberada, representado con un semáforo MUTEX, y otros dos semáforos que nos permiten saber si la barra tiene algún plato o no.

A su vez, llevamos contadores de los platos entregados, para poder finalizar el problema con la cantidad de platos requerida. Intentamos abstraer los métodos para retirar y cocinar platos, así se visualiza de una forma mas específica el manejo de semáforos y concurrencia en los métodos de cocinero y mozo.

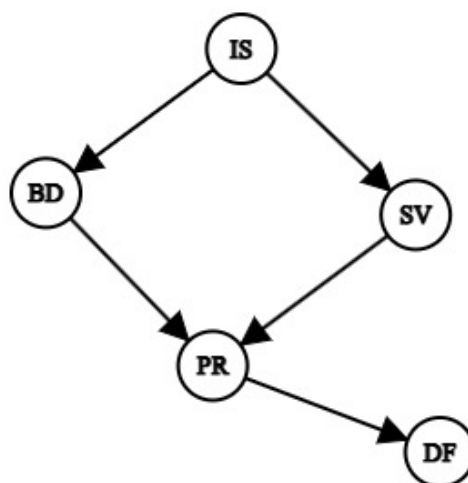
2.1.2. Ejecución

Para poder ver el correcto funcionamiento del ejercicio, debemos compilarlo y ejecutarlo. Una vez ejecutado se observa por consola la iteración entre los cocineros, la barra y los mozos. Cada ejecución es distinta, dado a que no podemos afirmar que hilo sera el precedente.

2.2. Ejercicio 2

2.2.1. Grafo de precedencias

Para este ejercicio, en base a las tareas y sus precedencias, diseñamos el siguiente grafo.



2.2.2. Inicialización de los semáforos

Para representar las dependencias se utiliza un semáforo por cada precedencia del grafo.

```
#include <semaphore.h>
sem_t sem_IS_BD; // Permite que BD inicie cuando termina IS
sem_t sem_IS_SV; // Permite que SV inicie cuando termina IS
sem_t sem_BD_PR; // Permite que PR inicie cuando termina BD
sem_t sem_SV_PR; // Permite que PR inicie cuando termina SV
sem_t sem_PR_DF; // Permite que DF inicie cuando termina PR
```

Cada semáforo representa la habilitación de la siguiente tarea cuando termina.

Luego, todos los semáforos se inicializan en 0. El valor inicial en 0 significa que la tarea dependiente esta bloqueada hasta que la tarea que la precede haga sem_post.

```
sem_init(&sem_IS_BD, 0, 0);
sem_init(&sem_IS_SV, 0, 0);
sem_init(&sem_BD_PR, 0, 0);
sem_init(&sem_SV_PR, 0, 0);
sem_init(&sem_PR_DF, 0, 0);
```

El valor inicial en 0 es correcto ya que inicio del programa todavía no se terminó ninguna tarea, por ende, ninguna dependencia está cumplida.

1) Tarea IS | Inicializar sistema

IS no depende de ninguna tarea, entonces puede ejecutarse directamente.

```
sem_post(&sem_IS_BD);
sem_post(&sem_IS_SV);
```

Al terminar, manda la señal a BD y a SV.

2) Tarea BD | Configurar base de datos

BD espera a que termine IS para poder ejecutarse.

```
sem_wait(&sem_IS_BD);
// Ejecuta BD
sem_post(&sem_BD_PR);
```

Cuando BD finaliza, avisa a PR, pero a PR todavía le falta otra dependencia.

3) Tarea SV | Configurar servidor

SV también espera a que termine IS para poder ejecutarse.

```
sem_wait(&sem_IS_SV);
// Ejecuta SV
sem_post(&sem_SV_PR);
```

Cuando SV finaliza, avisa a PR que esta dependencia fue completada.

4) Tarea PR | Ejecutar pruebas

PR tiene dos dependencias, BD y SV, por lo tanto, tiene que hacer un sem_wait a ambas tareas.

```
sem_wait(&sem_BD_PR);
sem_wait(&sem_SV_PR);
// Ejecuta PR
sem_post(&sem_PR_DF);
```

Cuando PR termina, avisa a DF, la ultima tarea

5) Tarea DF | Deploy final

DF solo depende de PR

```
sem_wait(&sem_PR_DF);
```

Al DF ser la ultima tarea, no tiene que avisarle a ninguna otra.

3. Parte 3

3.1. Idea inicial

La primera forma que buscamos para resolver el problema del juego en ADA, fue identificar cuales iban a ser nuestras tareas, y que iban a representar. Una vez encontramos en el juego cuales serian las tareas, nos enfocamos en saber si iban a ser únicas o no, es decir, sabíamos necesitaríamos una tarea para las balas, pero estas balas no iban a ser únicas es por esto que las definimos como un tipo de tarea.

3.2. Construcción del código

Comenzamos a armar un esqueleto con ayuda pseudocódigo visto en clase de practico, enfocando el Main en manejar el recibir los inputs del jugador y redirigir estos a los procedimientos necesarios.

3.3. Inteligencia Artificial

Una vez ya terminado ese esqueleto, nos ayudamos fuertemente con herramientas de IA, para poder lograr al menos una primera instancia del juego.

Ejemplo de prompt:

Quiero mantener la estructura del proyecto que se encuentra ahora mismo, utilizando tareas, entrys y procedures. Quiero que en el main mantengas esta parte del movimiento del usuario, pero luego las demás acciones si sean tareas. Quiero que utilices el Get_inmediate para poder seguir corriendo la consola, al mismo tiempo que se reciben inputs y outputs. Intenta seguir la lógica de ej3/ejepmlo.adb. Intenta no utilizar cosas complejas de ADA, y que todo se mantenga dentro del mismo archivo.

Iteramos varias veces con agente de IA, hasta que llegamos a un resultado optimo. También utilizamos el agente asegurarnos que nunca una tarea quede a de un llamado si tiene un accept. Dado a que con una extensión larga de código, podría llegar a ser muy probable que definamos una tarea con un entry que quede colgado.

3.4. Ejemplo de una tarea

```
task type Bala is
  entry Disparar (X : Integer; Y : Integer; Aceptada : out Boolean);
  entry Consultar (X : out Integer; Y : out Integer; Activa : Boolean);
  entry Apagar;
end Bala;
task body Bala is
  Pos_X : Integer := 1;
```

```
Pos_Y : Integer := 1;
Esta_Activa : Boolean := False;
Terminar : Boolean := False;
Ciclos_Movimiento : Integer := 0;
```

4. Parte 4

4.1. Elección de imagen base

Para el ejercicio 1, 2, 3 y el manager decidimos utilizar la imagen de:

```
FROM debian:bookworm-slim
```

Investigamos a través de IA, cual era la imagen mas liviana y que permitiera acceder a los recursos para ejecutar los ejercicios. En el ejercicio 2, que necesitábamos compilar en C, a su vez utilizamos la siguiente imagen:

```
FROM gcc:bookworm
```

4.2. Análisis de seguridad

Para esta parte tuvimos en cuenta que la letra pedía minimizar la superficie de ataque de los contenedores. Por esto mismo, la idea principal fue que los ejercicios no quedaran expuestos directamente desde la máquina host, sino que el único punto de entrada sea el contenedor manager.

Los contenedores ej1, ej2 y ej3 funcionan como runners. Estos no publican puertos hacia el host, sino que solamente aceptan conexiones SSH dentro de la red interna de Docker Compose. De esta forma, si queremos ejecutar alguno de los ejercicios, primero entramos al manager y desde ahí se ejecuta el comando SSH correspondiente.

El único puerto publicado hacia la máquina host es el 8080, usado por el manager para mostrar la interfaz web de monitoreo. Esto reduce la cantidad de servicios accesibles desde afuera del entorno de Docker.

También analizamos que no todos los contenedores podían tener el mismo nivel de restricción. En ej2 y ej3, como solamente se ejecutan binarios ya compilados, pudimos dejar el filesystem principal en modo solo lectura. En cambio, en ej1 no lo aplicamos porque el script de Bash puede modificar archivos del inventario y de la carpeta mercaderia.

Un punto importante de seguridad es el uso del socket de Docker en el manager:

```
/var/run/docker.sock:/var/run/docker.sock:ro
```

Esto permite que el manager pueda consultar métricas reales de los contenedores para mostrarlas en la interfaz web. Aun así, entendemos que el socket de Docker es un recurso sensible, por eso se montó en modo solo lectura y se utiliza solamente para consultar estado, CPU, memoria y red.

4.3. Configuración de seguridad aplicada

La primera medida aplicada fue ejecutar los procesos principales con usuarios no root. En los contenedores de los ejercicios se creó el usuario runner, mientras que en el manager se creó el usuario manager. Esto evita que las aplicaciones corran directamente como root dentro del contenedor.

Para el acceso SSH de los runners configuramos:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers runner
UsePAM no
```

Con esto no se permite iniciar sesión como root, no se permite autenticación por contraseña y solo se puede entrar usando clave pública. Además, solamente el usuario runner queda autorizado para conectarse.

En el archivo `docker-compose.yml` agregamos la siguiente configuración en los servicios:

```
cap_drop:
  - ALL
security_opt:
  - no-new-privileges:true
```

Con `cap_drop: ALL` se eliminan las capabilities por defecto que Docker suele darle a los contenedores. Esto reduce los permisos disponibles dentro del contenedor. Con `no-new-privileges:true` evitamos que un proceso pueda escalar privilegios mediante mecanismos como binarios `setuid`.

También usamos `tmpfs` para `/tmp`:

```
tmpfs:
  - /tmp
```

Esto deja disponible un espacio temporal en memoria sin tener que escribir en el filesystem principal del contenedor. Fue necesario porque varios servicios, como SSH o `nginx`, necesitan archivos temporales para funcionar.

En los contenedores `ej2` y `ej3` aplicamos además:

```
read_only: true
```

Esto hace que el filesystem principal sea de solo lectura. Lo pudimos hacer porque esos contenedores solamente ejecutan programas ya compilados y no necesitan modificar archivos persistentes.

En `ej1` no usamos `read_only: true`, ya que el ejercicio de Bash trabaja con archivos como `inventario_f1.csv` y los manifiestos dentro de `mercaderia`. En el manager tampoco lo aplicamos porque genera dinámicamente el archivo `status.json` que consume la interfaz web.

Por último, las claves SSH no se entregan dentro del zip. Se generan automáticamente con el servicio `one-shot ssh-init` cuando se ejecuta `docker compose up`.

```
ssh_private:/run/ssh/private:ro
ssh_authorized:/home/runner/.ssh:ro
```

El volumen `ssh_private` queda montado solamente en el manager y contiene la clave privada. El volumen `ssh_authorized` queda montado en los runners y contiene solamente `authorized_keys`, que usa la clave pública autorizada. De esta manera, cada máquina que levanta el proyecto genera sus propias claves y la clave privada no queda copiada dentro de los contenedores de los ejercicios.

4.4. Decisiones de diseño

Para diagramar el diseño de las imagenes, usamos la estrategia vista en la clase de practico. Mantener el manager con la parte del front, trabajando via SSH con las otras tres imagenes. El docker-compose se asegura de primero levantar los ejercicios como servicios para luego poder levantar el servicio del manager.

4.5. Inteligencia Artificial

Para realizar esta parte nos apoyamos fuertemente en agentes de IA, lo utilizamos para cubrir los detalles de seguridad que requerían la letra. Esta recomendó generar las claves publicas y privadas para que luego el manager pueda acceder mediante ssh a los distintos servicios.

Para la parte del frontEnd, delegamos completamente la creación del mismo y la captura de datos al agente de IA. Este utiliza un .json donde guarda los datos de los contenedores, luego con python los lee y los muestra en la interfaz, la interfaz esta hecha con NGNIX y no https.

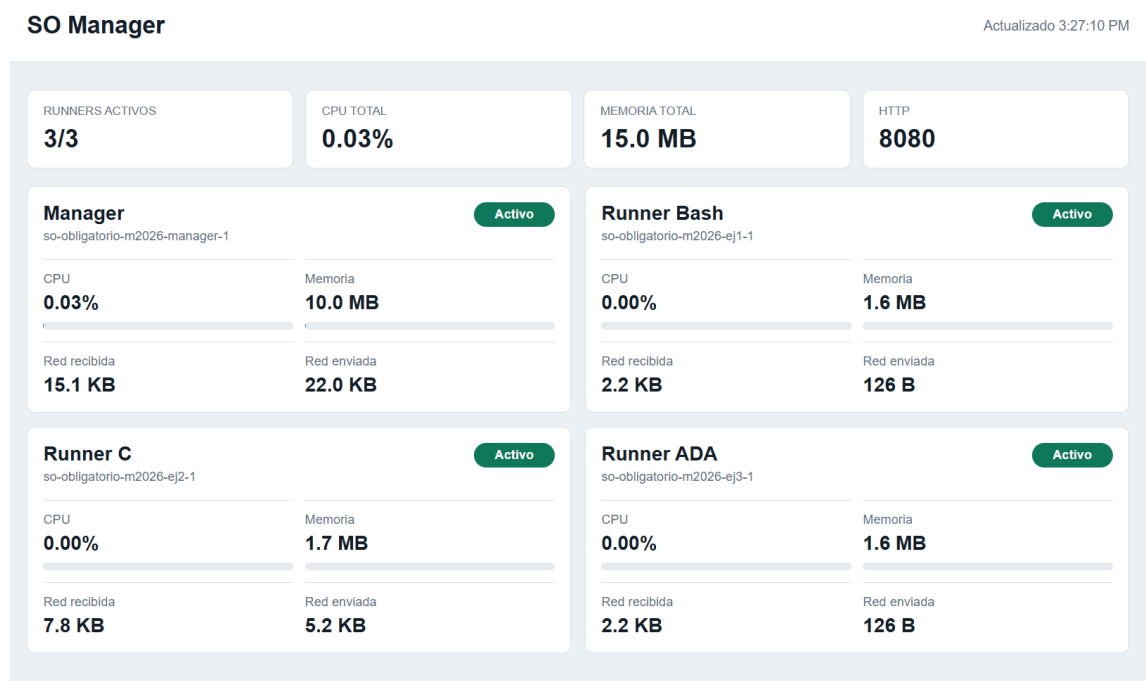
4.6. Ejecución

Para ejecutar debemos primero utilizar el comando:

```
docker-compose up
```

Este levanta todos los servicios y también nos crea las claves para que luego se puedan hacer las conexiones ssh. Si queremos ver el estado de los contenedor por la interfaz de usuario debemos ir a:

```
http://localhost:8080/
```



Si queremos ejecutar por terminal los ejercicios podemos hacerlo de la siguiente manera:

```
docker compose exec manager ssh ej1 'cd /app && ./paddock_manager_fixed.sh  
buscar "Gorra Edicion Especial Monza"  
docker compose exec manager ssh ej2 'cd /app && ./parte2_code'  
docker compose exec manager ssh ej3 'cd /app && ./main'
```